Ingatlan API — Postman-tesztek

Bemutató feladat · ötféle Postman-teszt lépésről lépésre, egy kész API-hoz

Most nem API-t írsz, hanem **teszteket** egy kész API-hoz. A teszt egy rövid program, amely egy kérés után megnézi a választ, és eldönti: az jött-e, aminek jönnie kellett. Ha egyszer megírtad, egy kattintással újra lefut — akárhányszor módosul az API.

1. Az API

Egy ingatlanhirdetéseket kezelő API-t kapsz. Négy végpontja van:

Végpont	Mit tesz?	Válasz
`GET /api/ingatlanok`	az összes ingatlan; `?varos=` paraméterrel csak egy városé	200
`GET /api/ingatlanok/:id`	egy ingatlan	200 · 400 · 404
`POST /api/ingatlanok`	új ingatlan; a `Location` fejléc az új elem címe	201 · 400
`DELETE /api/ingatlanok/:id`	egy ingatlan törlése	204 · 400 · 404

Egy ingatlan mezői: `id`, `varos`, `tipus` (`lakás` vagy `ház`), `alapterulet` (m²), `szobak`, `ar` (Ft). A hibaválaszok alakja: `{ "hiba": "…" }`.

Indítsd el: nyisd meg az **ingatlan-api** mappát a VS Code-ban, és a terminálban add ki:

```
npm install
npm run dev
```

Az API a `http://localhost:3000` címen fut, 12 ingatlannal. Az adatok a memóriában vannak: minden újraindításkor visszaáll a kezdeti 12 ingatlan.

2. Az ötféle teszt

#	Teszttípus	Mit ellenőriz?
1	Státuszkód	a válasz kódját: 200, 201, 404…
2	Fejléc	a válasz egy fejlécét, például a `Content-Type`-ot vagy a `Location`-t
3	Tartalom	a törzsben lévő értékeket: hány elem jött, mi a város, mennyi az ár
4	Séma	a törzs **alakját**: megvannak-e a mezők, és jó-e a típusuk
5	Láncolás	az egyik kérés eredményét elmenti, és a következő kérés felhasználja

3. A Postman előkészítése

1. Hozz létre egy gyűjteményt **Ingatlan API — tesztek** néven.
2. Vegyél fel egy környezetet **Ingatlan API — helyi** néven, benne egy `host` változót: `http://localhost:3000/api`. Jobb fent válaszd ki.

3. Minden kérést ebbe a gyűjteménybe ments el (**Ctrl + S**), a megadott néven.

A tesztek helye a kérés **Scripts** fülén a **Post-response** szakasz: ami ide kerül, az a válasz megérkezése után fut le. Egy teszt így néz ki:

```
pm.test("A teszt neve", () => {
  // itt az ellenőrzés
});
```

- `pm.test(név, függvény)` — egy teszt. A neve jelenik meg az eredmények között.
- A függvényben **állítások** állnak: „ennek ennyinek kell lennie”. Ha egy állítás nem teljesül, a teszt elbukik (piros); ha mind teljesül, átmegy (zöld).
- Az eredményt a válasz alatti **Test Results** fülön látod.

4. Első teszttípus: státuszkód

4.1 A lista

Készíts egy **GET** kérést a `{{host}}/ingatlanok` címre, és mentsd el **Lista** néven. A **Post-response** szakaszba:

```
pm.test("A státusz 200", () => {
  pm.response.to.have.status(200);
});
```

- `pm.response` — a megkapott válasz.
- `.to.have.status(200)` — az állítás: a válasz kódja 200. Ha más jön, a teszt elbukik.

Küldd el a kérést. A **Test Results** fülön megjelenik a teszt neve, zölden.

4.2 A hibás esetek

A jó tesztsor azt is megnézi, hogy az API jól mond-e **nemet**. Készíts még két kérést:

Név	Kérés	A teszt
Nem létező	`GET {{host}}/ingatlanok/99`	ugyanez, `404`-gyel
Hibás azonosító	`GET {{host}}/ingatlanok/abc`	ugyanez, `400`-zal

A teszt nevében és a `status(…)` zárójelében is írd át a kódot.

5. Második teszttípus: fejléc

5.1 A válasz JSON

A **Lista** tesztjei alá:

```
pm.test("A válasz JSON", () => {
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});
```

- `pm.response.headers.get("Content-Type")` — egy fejléc értéke szövegként, itt `application/json; charset=utf-8`.
- `pm.expect(érték)` — egy tetszőleges értékre tett állítás eleje.

- `.to.include("application/json")` — a szöveg **tartalmazza** ezt a részt. Pontos egyezés itt nem jó, mert a fejléc végén ott a `; charset=utf-8` is.

5.2 Az új ingatlan címe

Készíts egy **POST** kérést a `{{host}}/ingatlanok` címre, **Új ingatlan** néven. A **Body** → **raw** → **JSON** fülre:

```
{ "varos": "Eger", "tipus": "lakás", "alapterulet": 58, "szobak": 2, "ar": 36900000 }
```

A **Post-response** szakaszba:

```
pm.test("A státusz 201", () => {
  pm.response.to.have.status(201);
});

const uj = pm.response.json();

pm.test("A Location az új ingatlanra mutat", () => {
  pm.expect(pm.response.headers.get("Location")).to.eql(`/api/ingatlanok/${uj.id}`);
});
```

- `pm.response.json()` — a válasz törzse JavaScript-objektumként. Az `uj` változóba tesszük, mert a következő tesztek is használják.
- `.to.eql(…)` — pontos egyezés.
- A backtick közé írt szövegben a `${uj.id}` helyére az új ingatlan azonosítója kerül: így a teszt minden futtatáskor a **saját** új elemét ellenőrzi.

Minden elküldött POST egy új ingatlant hoz létre a memóriában, így a lista egyre hosszabb lesz. A 8. pontban a gyűjtemény maga törli, amit létrehozott; addig, ha a lista hossza zavar, indítsd újra az API-t.

6. Harmadik teszttípus: tartalom

6.1 Hány ingatlan jött?

A **Lista** tesztjei alá:

```
const lista = pm.response.json();

pm.test("Tömb, 12 ingatlannal", () => {
  pm.expect(lista).to.be.an("array");
  pm.expect(lista).to.have.lengthOf(12);
});
```

- `.to.be.an("array")` — a törzs tömb.
- `.to.have.lengthOf(12)` — pontosan 12 eleme van.
- Egy teszten belül több állítás is állhat; mindnek teljesülnie kell.

6.2 Szűrés városra

Új kérés: **GET** `{{host}}/ingatlanok?varos=Szeged`, **Szűrés városra** néven. A státusztesztje 200, alá:

```
const lista = pm.response.json();

pm.test("3 szegedi ingatlan", () => {
  pm.expect(lista).to.have.lengthOf(3);
  pm.expect(lista.every((i) => i.varos === "Szeged")).to.be.true;
});
```

- `lista.every(…)` — igaz, ha **minden** elemre igaz a feltétel. Így nemcsak a darabszámot nézzük, hanem azt is, hogy tényleg csak szegedi ingatlan jött.
- `.to.be.true` — az érték `true`.

6.3 Egy ingatlan adatai

Új kérés: **GET** `{{host}}/ingatlanok/3`, **Egy ingatlan** néven. A státusztesztje 200, alá:

```
const ingatlan = pm.response.json();

pm.test("A 3-as: budapesti ház", () => {
  pm.expect(ingatlan.id).to.eql(3);
  pm.expect(ingatlan.varos).to.eql("Budapest");
  pm.expect(ingatlan.tipus).to.eql("ház");
});
```

6.4 A hibaüzenet

A **Nem létező** kérés tesztjei alá:

```
pm.test("Van hibaüzenet", () => {
  pm.expect(pm.response.json().hiba).to.be.a("string");
});
```

- `.to.be.a("string")` — a `hiba` mező szöveg. A pontos szövegét nem nézzük: azt az API bármikor átfogalmazhatja, a tesztnek ettől még zöldnek kell maradnia.

Készíts egy **Hibás új ingatlan** nevű POST kérést is, ugyanarra a címre, ár nélküli törzzsel:

```
{ "varos": "Eger", "tipus": "lakás", "alapterulet": 58, "szobak": 2 }
```

Tesztjei: a státusz **400**, és ugyanez a „Van hibaüzenet” teszt.

6.5 Az új ingatlan adatai

Az **Új ingatlan** tesztjei alá:

```
pm.test("Visszakaptuk, amit küldtünk", () => {
  pm.expect(uj.varos).to.eql("Eger");
  pm.expect(uj.ar).to.eql(36900000);
});
```

Az `uj` változót az 5.2-ben már létrehoztad, itt csak felhasználod.

7. Negyedik teszttípus: séma

A tartalom-teszt konkrét értékeket néz. A séma-teszt az **alakot**: akármilyen ingatlan jön, legyen meg minden mezője, és mindegyik a jó típusú legyen.

Az **Egy ingatlan** tesztjei alá:

```
const ingatlanSema = {
  type: "object",
  required: ["id", "varos", "tipus", "alapterulet", "szobak", "ar"],
  properties: {
    id: { type: "integer" },
    varos: { type: "string" },
    tipus: { type: "string", enum: ["lakás", "ház"] },
    alapterulet: { type: "number" },
    szobak: { type: "integer" },
    ar: { type: "number" },
  },
};

pm.test("Ingatlan alakú", () => {
  pm.response.to.have.jsonSchema(ingatlanSema);
});
```

- `type: "object"` — a válasz egy objektum.
- `required` — ezeknek a mezőknek meg kell lenniük.
- `properties` — mezőnként a típus. Az `integer` egész szám, a `number` bármilyen szám.
- `enum` — csak a felsorolt értékek valamelyike lehet.
- `.to.have.jsonSchema(…)` — a válasz megfelel-e a sémának.

A **Lista** minden eleme ugyanilyen alakú. Másold át az `ingatlanSema` állandót a **Lista** tesztjei közé is, és alá:

```
pm.test("Minden elem ingatlan alakú", () => {
  pm.response.to.have.jsonSchema({ type: "array", items: ingatlanSema });
});
```

- `{ type: "array", items: … }` — a válasz tömb, és **minden** eleme megfelel a megadott sémának.

8. Ötödik teszttípus: láncolás

Az új ingatlant le is akarjuk kérni és törölni — ehhez kell az azonosítója, az viszont minden futtatáskor más. A megoldás: az **Új ingatlan** elmenti egy változóba, a későbbi kérések onnan veszik.

8.1 Az azonosító mentése

Az **Új ingatlan** tesztjei végére:

```
pm.collectionVariables.set("ujId", uj.id);
```

- `pm.collectionVariables.set(név, érték)` — gyűjteményváltozót állít be. Ez nem teszt, nincs `pm.test` körülötte: csak elment egy értéket.
- A változót a gyűjtemény **Variables** fülén is látod, a kérések címében pedig `{{ujId}}` alakban használhatod.

8.2 A lánc többi kérése

Az **Új ingatlan** és a **Hibás új ingatlan** után vedd fel ezt a három kérést, ebben a sorrendben:

Az új ingatlan — GET `{{host}}/ingatlanok/{{ujId}}`. A státusztesztje 200, alá:

```
pm.test("Ez az egri lakás", () => {
  pm.expect(pm.response.json().varos).to.eql("Eger");
});
```

Törlés — DELETE `{{host}}/ingatlanok/{{ujId}}`. A státusztesztje 204, alá:

```
pm.test("A törzs üres", () => {
  pm.expect(pm.response.text()).to.eql("");
});
```

- `pm.response.text()` — a törzs nyers szövegként. A 204-es válasznak nincs törzse, ezért itt nem a `json()`-t hívjuk.

Törlés után — GET `{{host}}/ingatlanok/{{ujId}}`. Egyetlen teszt: a státusz **404**.

9. Az egész gyűjtemény futtatása

A gyűjteményben a kérések sorrendje és a tesztek száma:

#	Név	Kérés	Teszt
1	Lista	`GET {{host}}/ingatlanok`	4
2	Szűrés városra	`GET {{host}}/ingatlanok?varos=Szeged`	2
3	Egy ingatlan	`GET {{host}}/ingatlanok/3`	3
4	Nem létező	`GET {{host}}/ingatlanok/99`	2
5	Hibás azonosító	`GET {{host}}/ingatlanok/abc`	1
6	Új ingatlan	`POST {{host}}/ingatlanok`	3
7	Hibás új ingatlan	`POST {{host}}/ingatlanok`	2
8	Az új ingatlan	`GET {{host}}/ingatlanok/{{ujId}}`	2
9	Törlés	`DELETE {{host}}/ingatlanok/{{ujId}}`	2
10	Törlés után	`GET {{host}}/ingatlanok/{{ujId}}`	1

A sorrendet a gyűjteményen belül húzással állíthatod be.

1. Indítsd újra az API-t, hogy a kezdeti 12 ingatlannal induljon.
2. A gyűjtemény **Run** gombjával nyisd meg a Runnert, és indítsd el a futtatást.
3. Az eredmény: 10 kérés, **22 teszt**, mind zöld.
4. Futtasd le még egyszer, újraindítás nélkül. Megint mind a 22 zöld.

A második futtatás azért zöld, mert a gyűjtemény **maga után takarít**: a 9. kérés törli, amit a 6. létrehozott, így a lista hossza a futtatás végén megint 12. Egy tesztsor akkor jó, ha akárhányszor lefuttatható, és mindig ugyanazt mondja.

10. Sorvezető

Amit használsz	Mit csinál?
`pm.test("név", () => { … })`	egy teszt
`pm.response.to.have.status(kód)`	a válasz kódja ennyi
`pm.response.headers.get("Név")`	egy válaszfejléc értéke
`pm.response.json()`	a törzs objektumként
`pm.response.text()`	a törzs nyers szövegként
`pm.expect(x).to.eql(y)`	x pontosan egyenlő y-nal
`pm.expect(x).to.include(y)`	x tartalmazza y-t
`pm.expect(x).to.be.an("array")`	x tömb
`pm.expect(x).to.be.a("string")`	x szöveg
`pm.expect(x).to.have.lengthOf(n)`	x hossza `n`
`pm.expect(x).to.be.true`	x értéke `true`
`pm.response.to.have.jsonSchema(séma)`	a törzs megfelel a sémának
`pm.collectionVariables.set("név", érték)`	gyűjteményváltozó mentése; `{{név}}` alakban használható

11. Ha nem működik

Tünet	Mi lehet a baj?
a kérés el sem megy, egyetlen teszt sem zöld	nem fut az API, vagy nincs kiválasztva a környezet
`JSONError` a **Törlés** tesztjénél	a 204-es válasz törzsére a `json()`-t hívtad — itt a `text()` kell
a „Tömb, 12 ingatlannal” elbukik: `but got 13`	előtte kézzel is elküldtél egy POST-ot — indítsd újra az API-t
a `{{ujId}}`-os kérések 400-at adnak	az **Új ingatlan** még nem futott le, vagy nem menti el az azonosítót
`ReferenceError: ingatlanSema is not defined`	a sémát csak az egyik kéréshez írtad meg; másold át a másikba is
a POST 400-at ad, pedig jó a törzs	a törzs nem **raw** → **JSON** típusú

12. Próbáld ki

Mindegyiknél nézd meg, mit ír a bukott teszt, utána állítsd vissza.

1. A **Lista** státusztesztjében írd át a `200`-at `201`-re. Az üzenet: `expected response to have status code 201 but got 200`.
2. Küldd el kézzel az **Új ingatlan** kérést, aztán futtasd le a gyűjteményt. A „Tömb, 12 ingatlannal” elbukik: `expected [ …(13) ] to have a length of 12 but got 13`. Indítsd újra az API-t, és megint minden zöld.

3. Az **Egy ingatlan** sémájában vedd fel a `required` listába az `"erkely"` mezőt. Az üzenet: `data should have required property 'erkely'`.
4. Ugyanott az `ar` típusát írd át `"string"`-re. Az üzenet: `data.ar should be string`.
5. Állítsd le az API-t, és futtasd le a gyűjteményt. Egyetlen teszt sem lesz zöld.

13. Önálló feladatok

Ezekhez nincs kész kód — az eddigiek alapján meg tudod írni őket.

1. **Kisbetűs szűrés.** Új kérés: `?varos=szeged`. Ugyanaz a 3 ingatlan jöjjön, mint nagybetűvel.
2. **Hibás típus.** Új POST kérés `"tipus": "kastély"` értékkel. A státusz 400 legyen, és legyen hibaüzenet.
3. **Válaszidő.** Egy hatodik teszttípus: a **Lista** válasza érkezzen meg 500 ms-on belül. A válaszidőt a `pm.response.responseTime` adja meg ezredmásodpercben; az állítás: `.to.be.below(…)`.

14. Ellenőrzőlista

1. A gyűjteményben 10 kérés van, a 9. pont sorrendjében.
2. Mind az ötféle teszt szerepel benne: státuszkód, fejléc, tartalom, séma, láncolás.
3. A Runnerben 22 teszt fut le, mind zöld.
4. Újraindítás nélkül másodszor lefuttatva is mind a 22 zöld.
5. A hibás esetekre (404, 400) is van teszt.